SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – pseudocode Design

Course Code:	**DSA0405**	**Course Title:**	**FUNDAMENTALS OF DATA SCIENCE**
CO Assessed:	**CO5 –Pseudocode Design**	**Assessment:**	**Assessment Tool 1– Pseudocode Design**
Weightage:	**30% of CIA**	**Total Marks:**	**100**
CO5 Statement:	To understand the concept of supervised and unsupervised methods in machine learning		
Student Name:	V.P Sneha	**Reg. No.:**	192324284
Section / Batch:	Slot-D/AI&DS	**Date:**	25/08/2026

Code of Conduct

I V.P Sneha (192324284) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: v.p sneha

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	**25**	**Thorough understanding; clearly identifies all key issues**	**Good understanding with most issues identified**	**Basic understanding; some issues missed**	**Poor understanding; problem not identified correctly**	
Application of Concepts	**25**	**Effectively applies relevant concepts to analyze the case deeply**	**Applies appropriate concepts with minor gaps**	**Limited application of concepts**	**Fails to apply relevant concepts**	
Analysis	**20**	**In-depth analysis with strong reasoning and insights**	**Good analysis with logical reasoning**	**Basic analysis with limited depth**	**Weak or no analysis; lacks reasoning**	

Solution Design	**20**	**Innovative, feasible solutions with strong justification**	**Logical solutions with adequate justification**	**Basic solutions with weak justification**	**Incorrect or no solution provided**	
Clarity	**10**	**Clear, well-structured, and easy to understand**	**Organized and understandable presentation**	**Limited clarity and structure**	**Poor clarity; difficult to understand**	

1. Supervised Learning

Problem Statement

Design a supervised learning workflow that accepts a labelled dataset, separates the features and target, trains a machine learning model using the training data, and predicts the target value for new data.

Algorithm

1. Start.
2. Read the labelled dataset.
3. Separate the input features X and target variable Y.
4. Split the dataset into training and testing data.
5. Select a supervised learning model.
6. Train the model using training features and target values.
7. Give new data as input.
8. Predict the target value using the trained model.
9. Display the prediction.
10. Stop.

Pseudocode

```
BEGIN
READ labelled_dataset
```

X ← Extract features from labelled_dataset

Y ← Extract target values from labelled_dataset

(X_train, X_test, Y_train, Y_test) ←

Split dataset into training and testing sets

model ← Create supervised learning model

Train model using X_train and Y_train

new_data ← Read new input data

prediction ← model.predict(new_data)

PRINT "Predicted Target:", prediction

END

Sample Output

Predicted Target: Pass

2. KNN Classifier – Majority Voting

Problem Statement

Design pseudocode for a **k-Nearest Neighbors (kNN)** classifier that calculates the distance between a test sample and all training samples, selects the k nearest samples, and predicts the class using majority voting.

Algorithm

1. Start.
2. Read the training dataset and test sample.
3. Select the value of k.
4. Calculate the distance between the test sample and every training sample.
5. Store the distance and corresponding class.
6. Sort the samples according to distance.
7. Select the first k nearest samples.
8. Count the occurrence of each class.
9. Select the class with the highest count.
10. Display the predicted class.
11. Stop.

Pseudocode

```
BEGIN
READ training_data
 READ test_sample
 READ k
 FOR each training_sample in training_data DO
 distance ← Calculate_Distance(test_sample, training_sample)
  Store(distance, training_sample.class)
   END FOR
   Sort all samples by distance in ascending order
   nearest_samples ← Select first k samples
   FOR each sample in nearest_samples DO
   Count the class of sample
   END FOR
   predicted_class ← Class with maximum count
   PRINT "Predicted Class:", predicted_class
   END
```

Sample Output

Enter k: 3

Nearest Classes:

Class A

Class A

Class B

Predicted Class: Class A

3. KNN Classifier – Student Pass/Fail

Problem Statement

Design pseudocode to classify a new student as Pass or Fail using kNN based on attendance percentage, internal marks, and assignment scores.

Algorithm

1. Start.
2. Read the student training dataset.
3. Each student contains:
    - Attendance percentage
    - Internal marks
    - Assignment score
    - Pass/Fail class
4. Read the new student's values.
5. Select the value of k.
6. Calculate the distance between the new student and every training student.
7. Sort the distances.
8. Select the k nearest students.
9. Count Pass and Fail among the nearest students.
10. If Pass count is greater, classify as Pass; otherwise classify as Fail.
11. Display the result.
12. Stop.

Pseudocode

```
BEGIN
  READ training_students
  READ new_attendance
  READ new_internal
  READ new_assignment
  READ k
  FOR each student in training_students DO
    distance ← SQRT(
      (student.attendance - new_attendance)^2 +
      (student.internal - new_internal)^2 +
      (student.assignment - new_assignment)^2
```

```
    )
    Store(distance, student.result)
  END FOR
  Sort students by distance in ascending order
  nearest_students ← Select first k students
  pass_count ← 0
  fail_count ← 0
  FOR each student in nearest_students DO
    IF student.result = "Pass" THEN
    pass_count ← pass_count + 1
 ELSE
 fail_count ← fail_count + 1
 END IF
 END FOR
 IF pass_count > fail_count THEN
 prediction ← "Pass"
 ELSE
prediction ← "Fail"
 END IF
PRINT "Student Result:", prediction
END
```

Sample Output

```
Enter Attendance: 85
Enter Internal Marks: 78
Enter Assignment Score: 90
Enter k: 3
Pass Count: 2
Fail Count: 1
Student Result: Pass
```

4. kNN Classifier – Selecting Best k

Problem Statement

Design pseudocode for selecting the appropriate value of **k** in a kNN classifier by evaluating different k values on validation data and selecting the value that gives the highest accuracy.

Algorithm

1. Start.
2. Read training and validation datasets.
3. Define possible values of k.
4. For each value of k:
    - Train/use kNN with that k.
    - Predict the validation data.
    - Calculate accuracy.
5. Compare the accuracies.
6. Select the k with the highest accuracy.
7. Display the best k.
8. Stop.

Pseudocode

```
BEGIN
  READ training_data
  READ validation_data
  k_values ← [1, 3, 5, 7, 9]
  best_k ← 0
  best_accuracy ← 0
  FOR each k in k_values DO
    correct ← 0
    total ← Number of validation samples
    FOR each sample in validation_data DO
      prediction ← kNN_Predict(
        training_data,
```

```
            sample,
            k
        )
        IF prediction = sample.actual_class THEN
            correct ← correct + 1
        END IF
    END FOR
    accuracy ← (correct / total) × 100
    PRINT "k =", k, "Accuracy =", accuracy
    IF accuracy > best_accuracy THEN
        best_accuracy ← accuracy
        best_k ← k
    END IF
  END FOR
  PRINT "Best k:", best_k
  PRINT "Best Accuracy:", best_accuracy
END
```

Sample Output

k = 1 Accuracy = 80%

k = 3 Accuracy = 90%

k = 5 Accuracy = 95%

k = 7 Accuracy = 92%

k = 9 Accuracy = 88%

Best k: 5

Best Accuracy: 95%

5. Decision Tree Classifier

Problem Statement

Design pseudocode to construct a **Decision Tree classifier** by selecting the best feature for splitting the training dataset and recursively creating child nodes until the stopping condition is reached.

Algorithm

1. Start.
2. Read the training dataset.
3. Check whether all samples belong to the same class.
4. If yes, create a leaf node with that class.
5. If no, calculate the quality of each available feature using a splitting measure such as **Information Gain**.
6. Select the feature with the highest Information Gain.
7. Create a decision node using the selected feature.
8. Split the dataset according to the feature values.
9. Recursively construct child nodes for each subset.
10. Stop splitting when all samples belong to the same class or no useful feature remains.
11. Return the Decision Tree.
12. Stop.

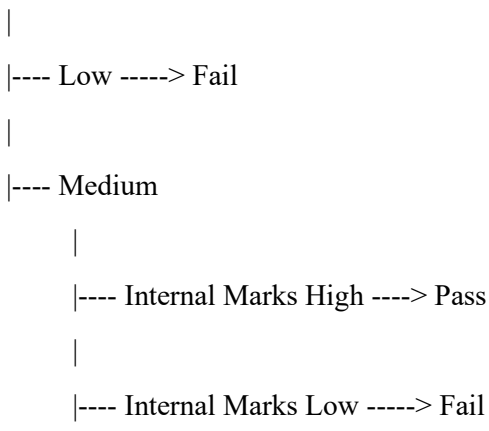
Pseudocode

```
BEGIN

FUNCTION BuildTree(dataset, features)

   IF all samples have the same class THEN

      RETURN LeafNode(class)

   END IF

   IF features is empty THEN

      RETURN LeafNode(MajorityClass(dataset))

   END IF

   best_feature ← NULL

   best_gain ← 0

   FOR each feature in features DO

      gain ← Calculate_Information_Gain(

                dataset,

                feature

             )

      IF gain > best_gain THEN
```

```
        best_gain ← gain
        best_feature ← feature
      END IF
    END FOR
    node ← Create Decision Node(best_feature)
    FOR each value of best_feature DO
      subset ← Select samples having this value
      child ← BuildTree(
                subset,
                features - best_feature
              )
      Add child to node
    END FOR
    RETURN node
END FUNCTION
    READ training_dataset
    features ← Extract features from dataset
    DecisionTree ← BuildTree(
                    training_dataset,
                    features
                  )
    PRINT DecisionTree
END
```

Sample Output

```
Best Feature: Attendance
Decision Tree:
Attendance
  |
  |---- High ----> Pass
```


|
|---- Low -----> Fail
|
|---- Medium
|
|---- Internal Marks High ----> Pass
|
|---- Internal Marks Low -----> Fail